

UNIVERSITETET I OSLO

Forsterkende Læring
(eng: Reinforcement
Learning)

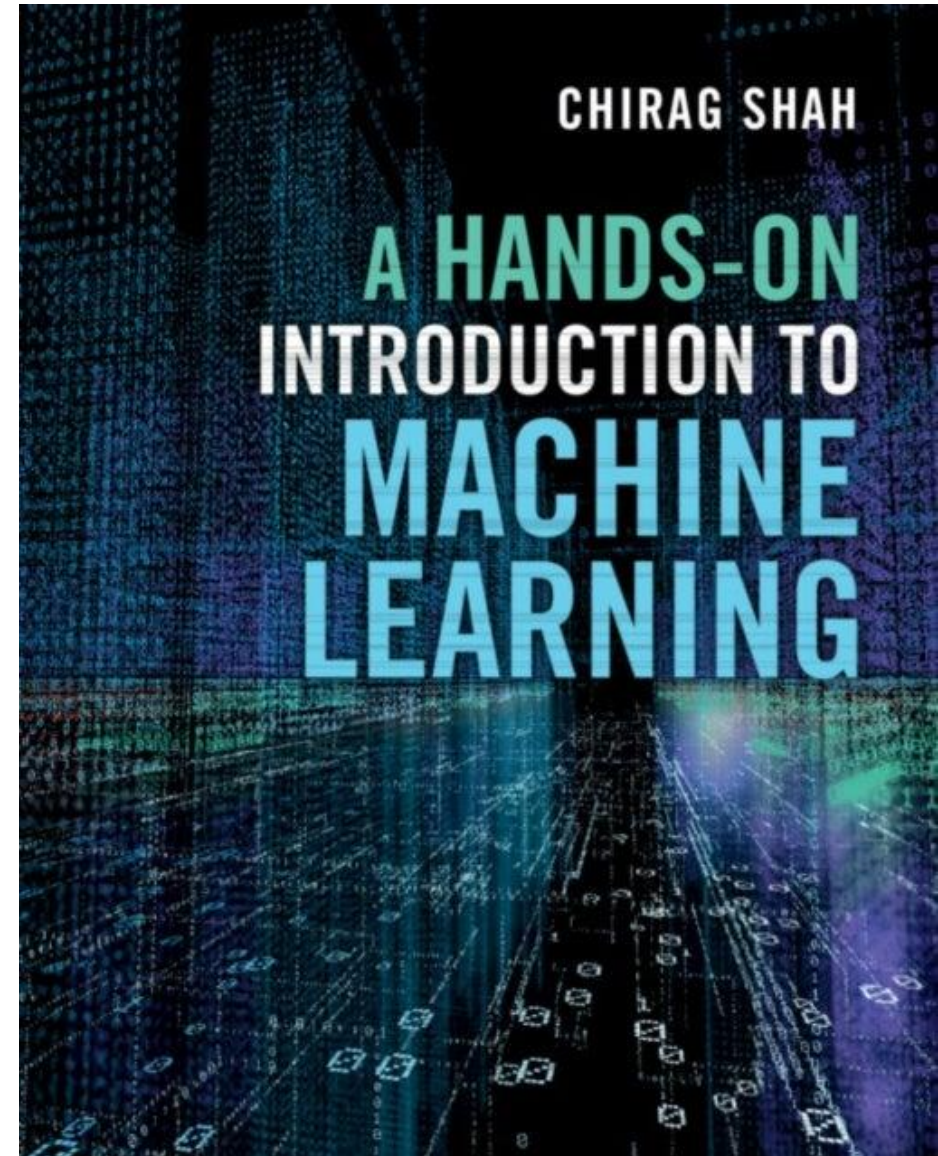
Kai Olav Ellefsen
*Gruppe for Robotikk og Intelligente
Systemer (ROBIN)*

IN1160 – Introduksjon til
maskinlæring (Vår 2026)



Pensum

- Kap 11 i Shah
- Og disse slidesene



Multi-Agent Hide and Seek

Veiledet læring

- Lær fra data med gitt fasit



-



?

Ikke-veildet læring

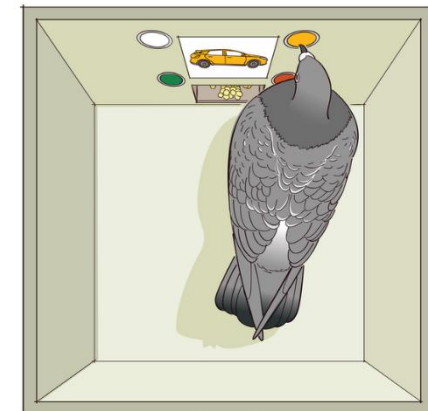
- Læring uten fasit



- Finn likheter og grupper data sammen

Forsterkende læring

- Trening med prøving og feiling, belønning og straff



Kilde: Wikipedia



Image by Steve Parker:

<https://search.creativecommons.org/photos/276926da-9c82-416f-89a0-b8809b4f1353>

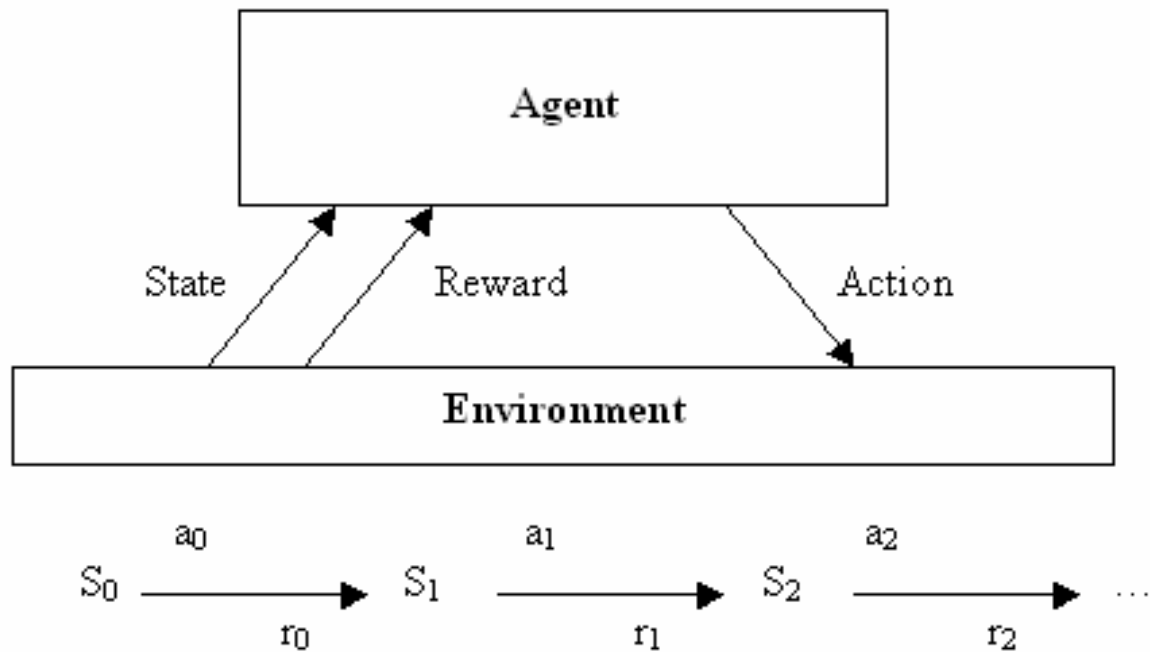


Noen ting å tenke på:

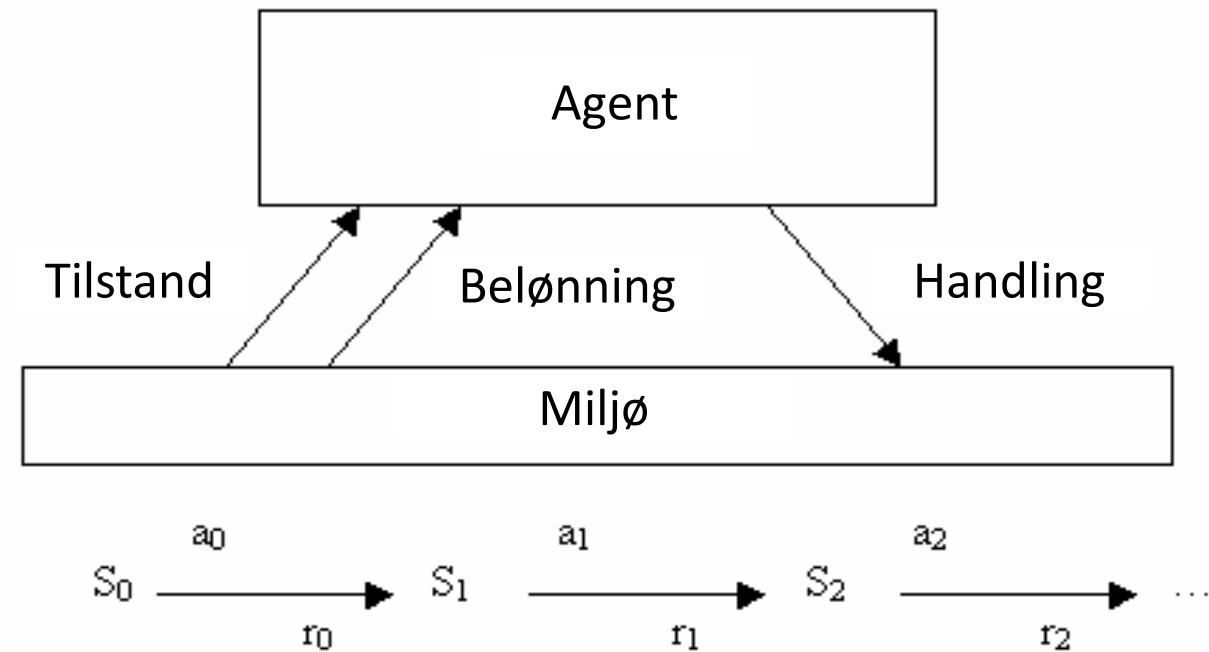
Hvordan vet hunden hvilke handlinger som førte til belønningen den fikk?

Hvordan kan hunden lære uten et “feilsignal” (forskjellen mellom det den gjorde og et fasitsvar)?

Forsterkende læring - problemformulering

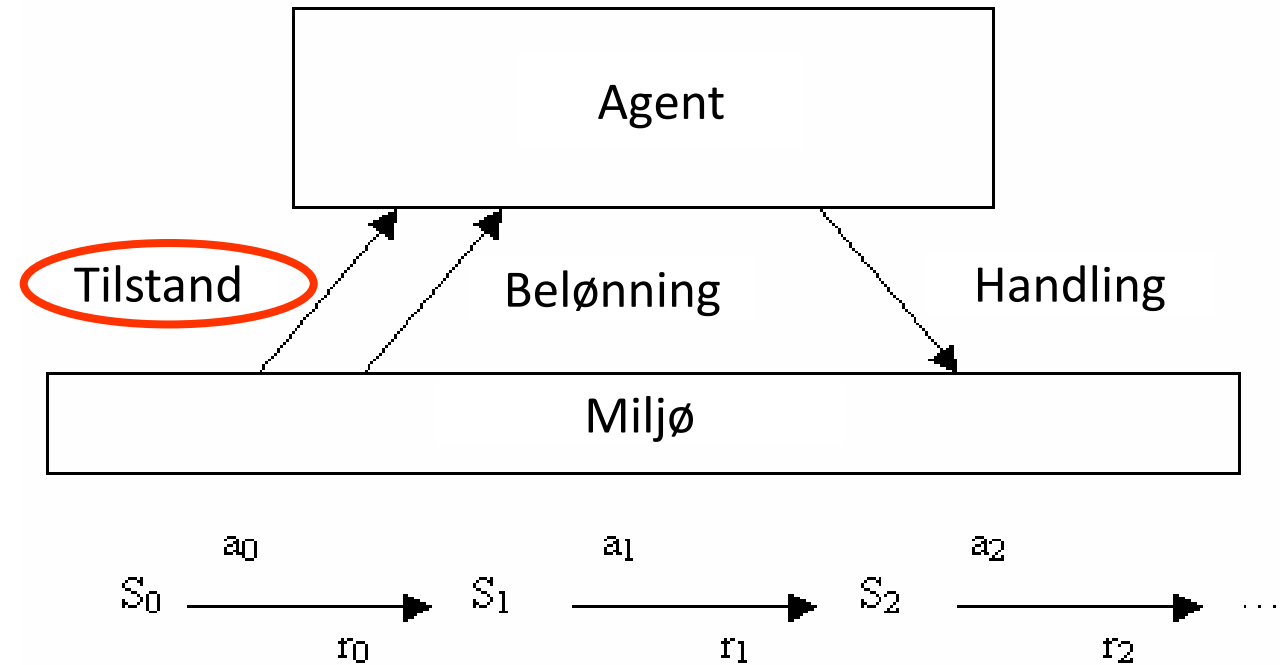


Goal: learn to choose actions that maximize:
 $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, where $0 \leq \gamma < 1$



Goal: learn to choose actions that maximize:
 $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, where $0 \leq \gamma < 1$

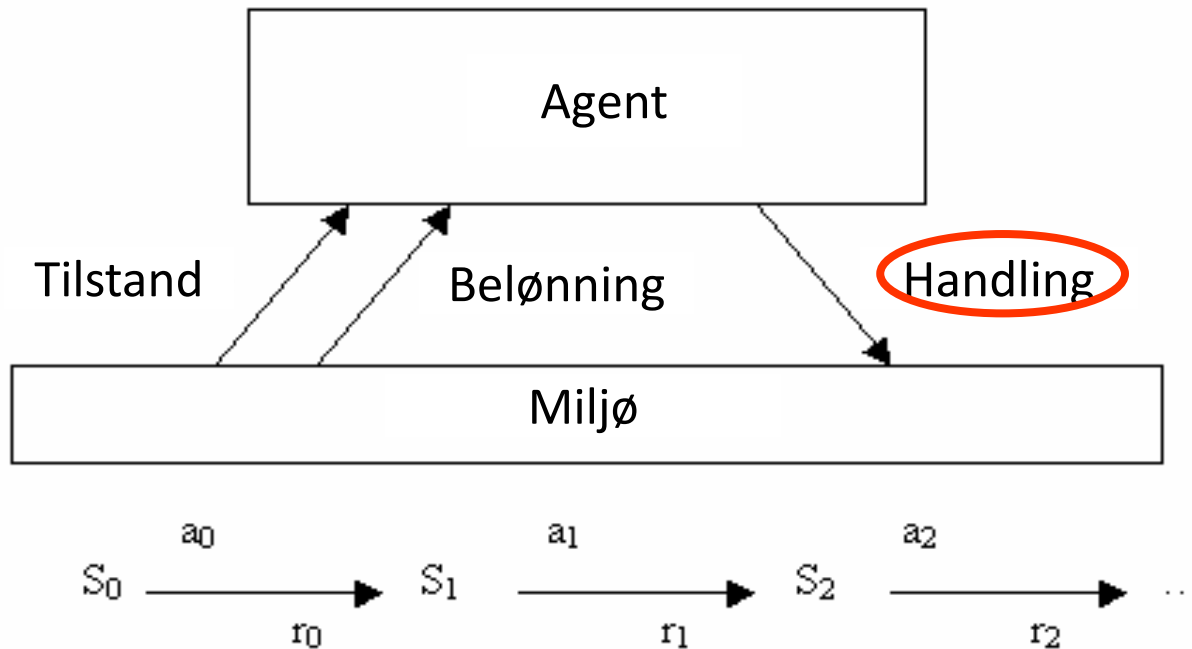
Forsterkende læring - problemformulering



Goal: learn to choose actions that maximize:
 $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, where $0 \leq \gamma < 1$



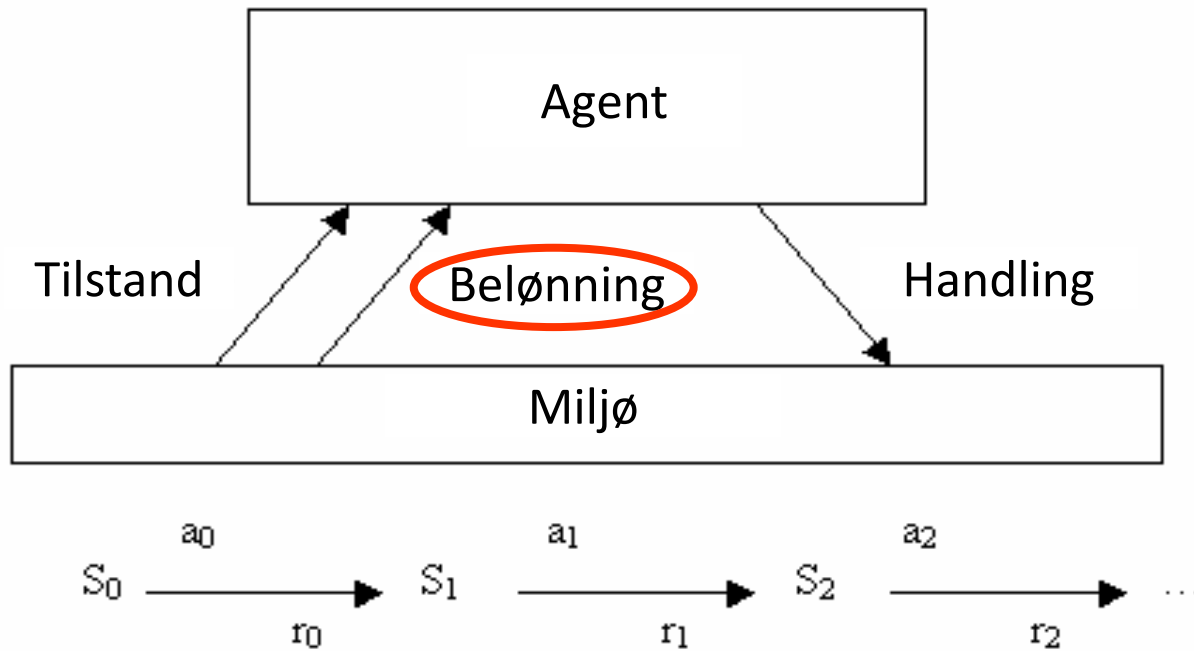
Forsterkende læring - problemformulering



“Flytt brikke fra J1 til H1”

Goal: learn to choose actions that maximize:
 $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, where $0 \leq \gamma < 1$

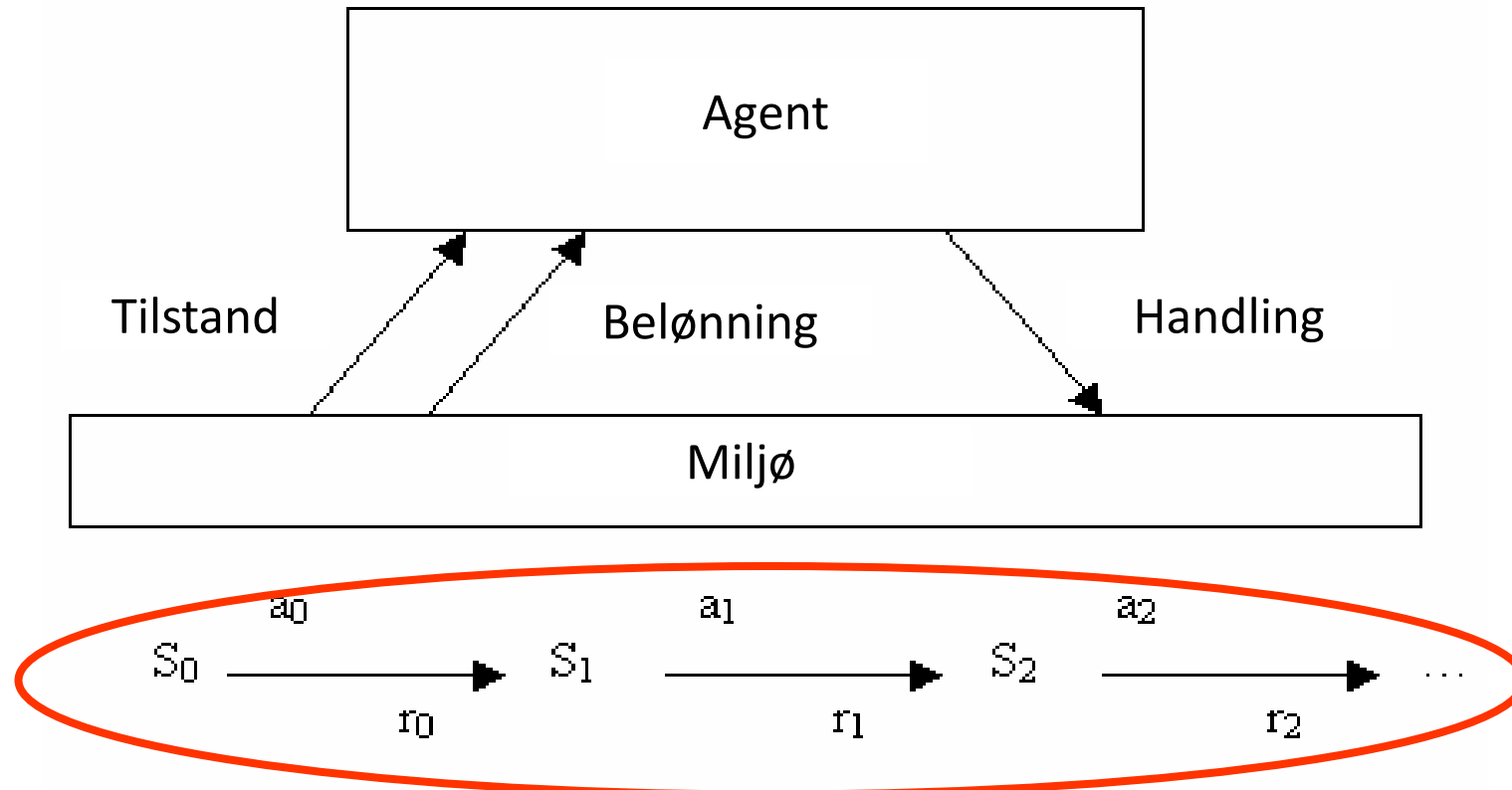
Forsterkende læring - problemformulering



Du tok motstanderens bonde.
Belønning=1

Goal: learn to choose actions that maximize:
 $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$, where $0 \leq \gamma < 1$

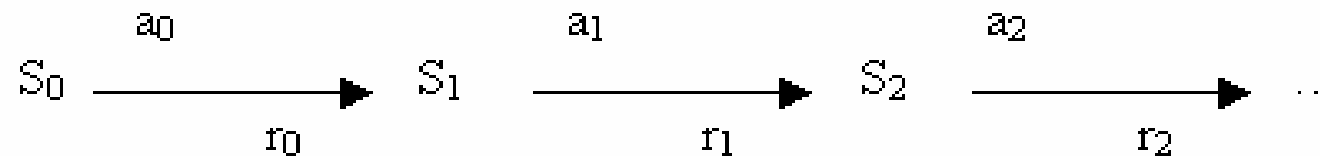
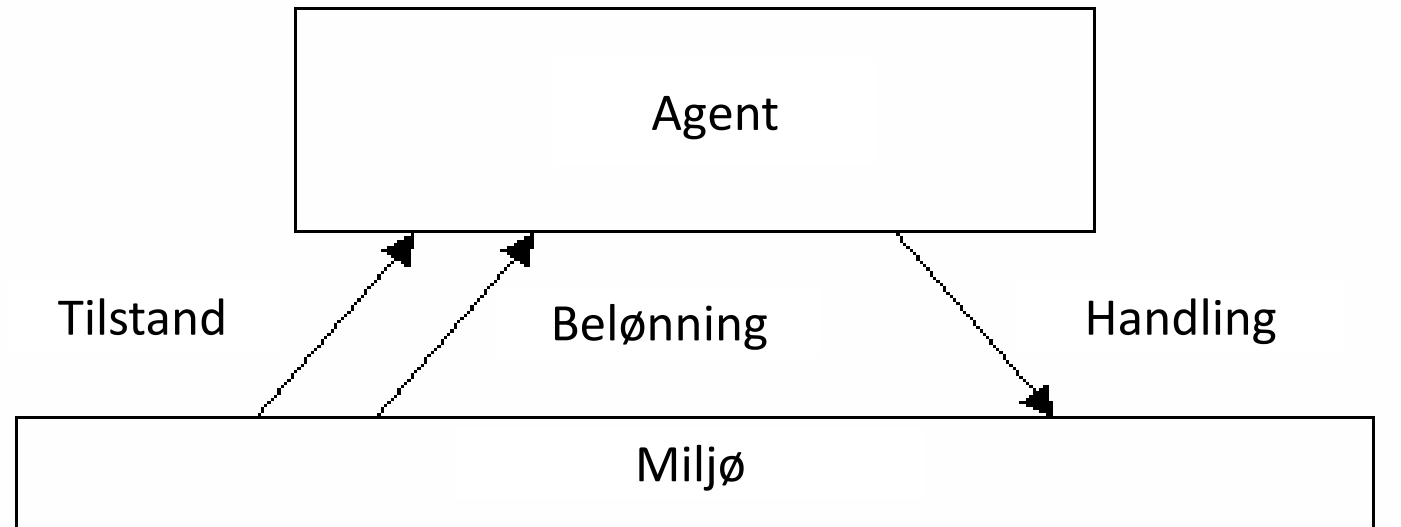
Forsterkende læring - problemformulering



Goal: learn to choose actions that maximize:

$$r_0 + \gamma r_1 + \gamma^2 r_2 + \dots, \text{ where } 0 \leq \gamma < 1$$

Forsterkende læring - problemformulering



Goal: learn to choose actions that maximize:

$$r_0 + \gamma r_1 + \gamma^2 r_2 + \dots, \text{ where } 0 \leq \gamma < 1$$

Læring styres av **belønningen**

- En sporadisk tilbakemelding (et tall) som sier hvor bra vi gjør det.
- Utfordringer:
 - Belønningen forteller ikke *hva vi burde ha gjort annerledes*
 - Belønningen kan være *forsinket* – det kan være vanskelig å vite *når vi gjorde feilen* (eng: *credit assignment problem*)

Belønningsfunksjonen

- r_{t+1} er en funksjon av (s_t, a_t)
- Defineres vanligvis manuelt av forskeren/utvikleren. **Krever prøving og feiling.**
- Belønningen er et tall. Kan være negativt (“straff”).
- Den kan gis flere ganger gjennom læringsepisoden eller kun helt til slutt.
- Agentens mål: *Å maksimere den totale belønningen.*

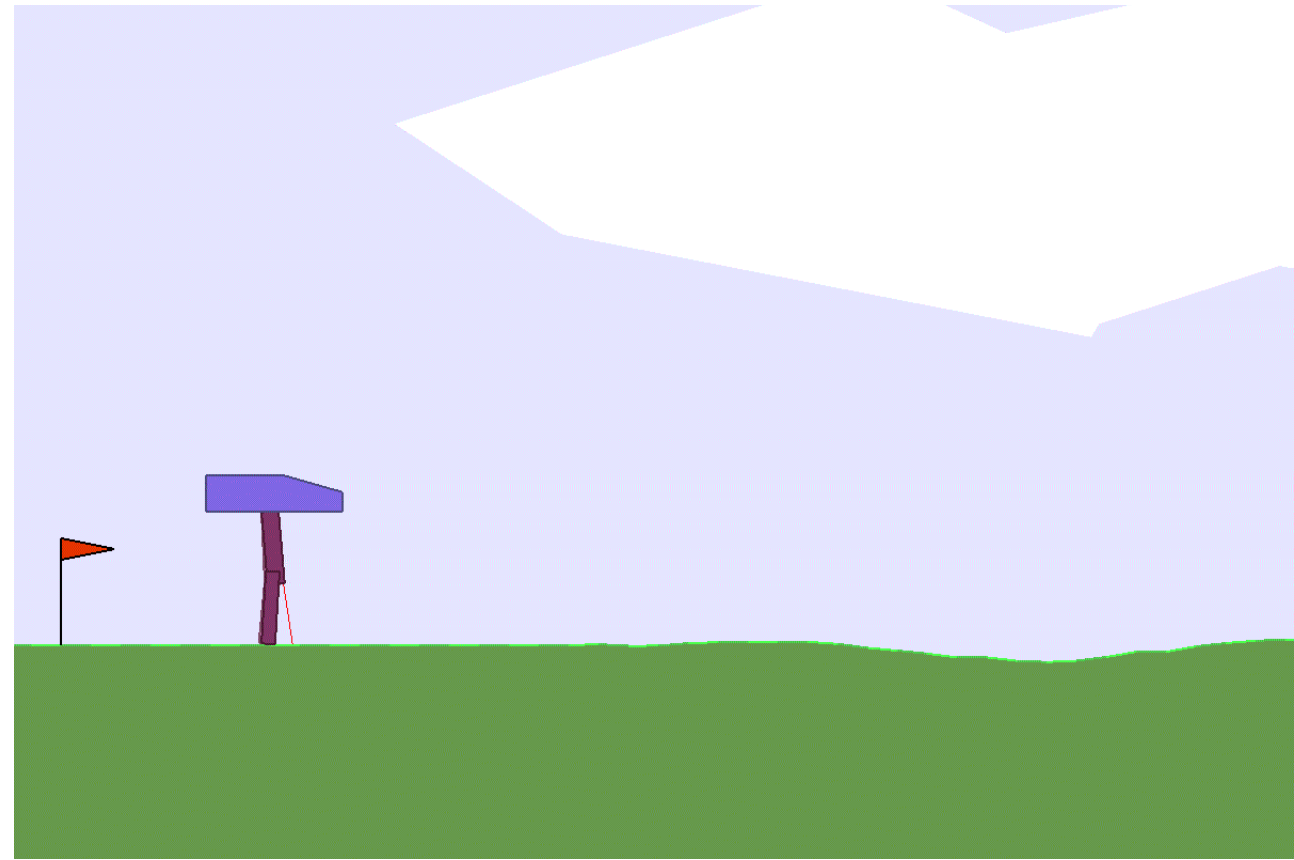


Belønningsfunksjoner kan være vanskelige å spesifisere.

Dårlig spesifiserte belønningsfunksjoner

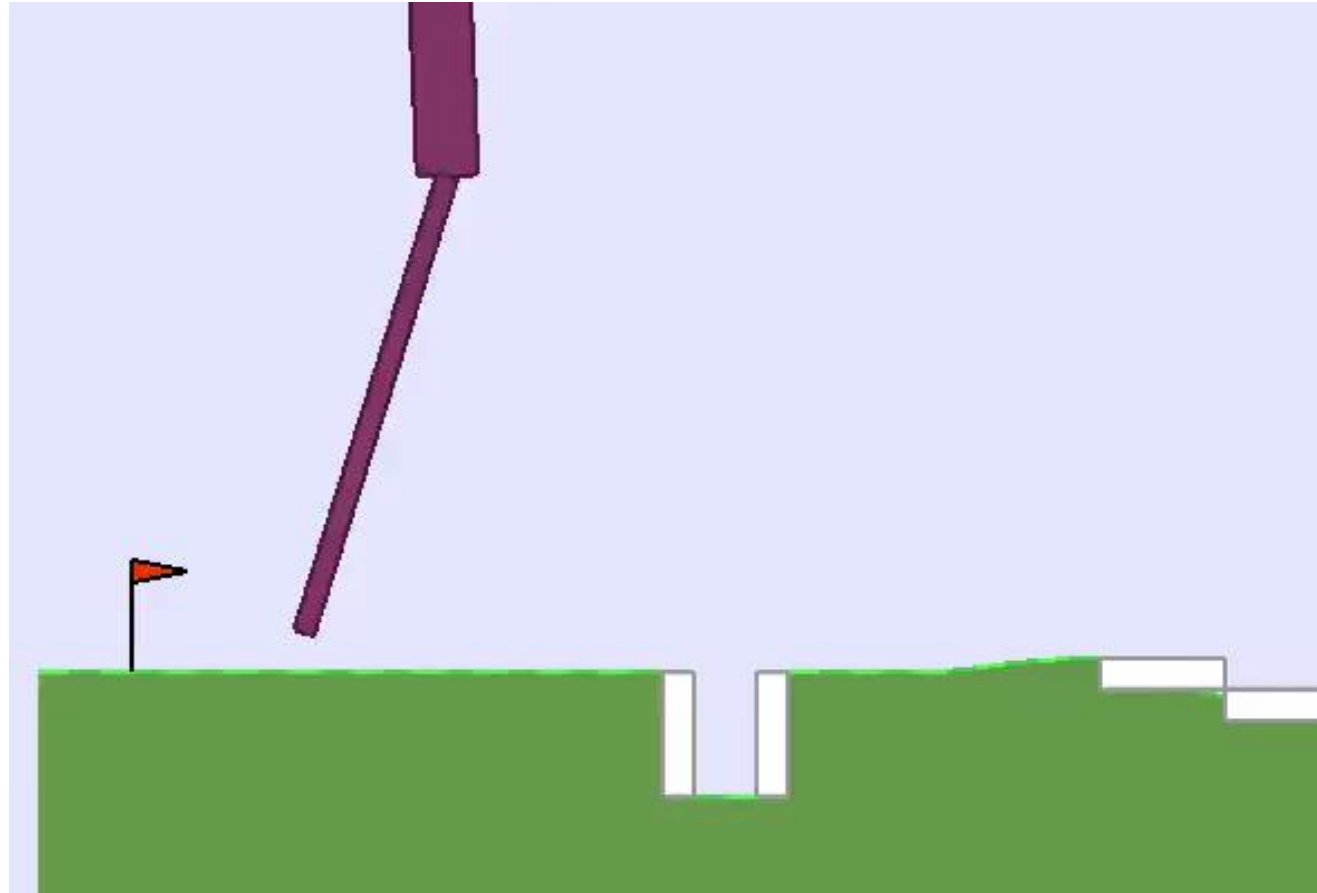
Se for dere at vi sier til algoritmen vår: «Gi belønning for å gå så langt som mulig på 10 sekunder. Du kan fritt justere størrelse og form på robot-beina og hvordan roboten går.»

Hva slags robot får vi?



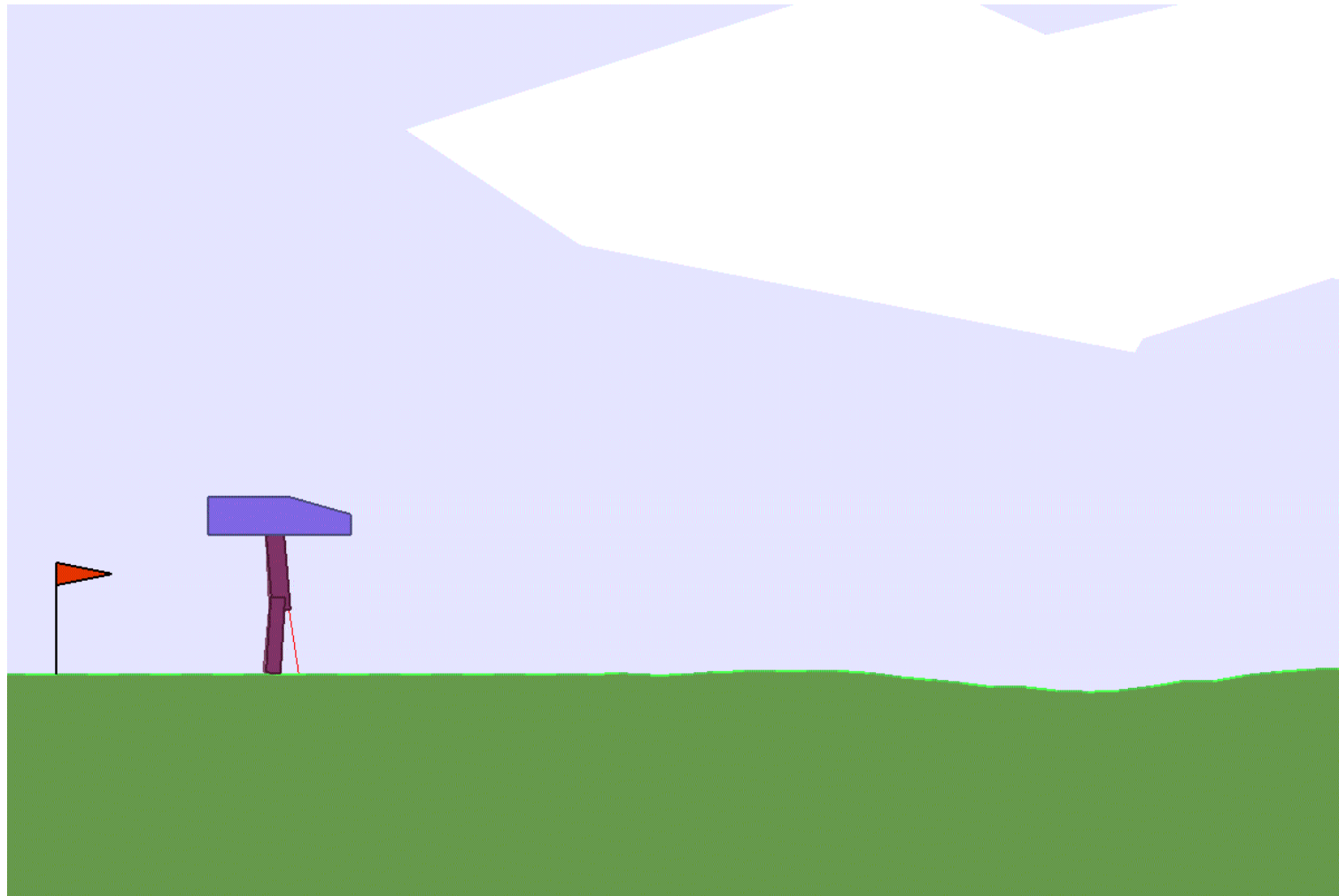
Source: OpenAI Gym

Dårlig spesifiserte belønningsfunksjoner



Source: David Ha
<https://designrl.github.io/>

Hvordan kan vi unngå dette? Hva kunne vært en bedre belønningsfunksjon?



Maksimere total belønning

- Total belønning:

$$R = \sum_{t=0}^{N-1} r_{t+1}$$

- Fremtidige belønninger kan være usikre – vi bryr oss derfor typisk mer om belønninger i nær fremtid
- Løsning: Å diskontere (eng: discount) fremtidige belønninger avhengig av hvor langt inn i fremtiden de er.

$$R = \sum_{t=0}^{\infty} \gamma^t r_{t+1}, \quad 0 \leq \gamma \leq 1$$

Diskonterte belønninger - eksempel

$$R = \sum_{t=0}^{\infty} \gamma^t r_{t+1}, \quad 0 \leq \gamma \leq 1$$

t	0.99^t	0.95^t
1	0.99	0.95
2	0.9801	0.9025
4	0.960596	0.814506
8	0.922745	0.66342
16	0.851458	0.440127
32	0.72498	0.193711
64	0.525596	0.037524

Diskonterte belønninger - eksempel

Kan dere tenke dere en situasjon der vi ville ønske å ha

a) Sterkt diskonterte fremtidige belønninger?

b) Lite diskonterte fremtidige belønninger?

t	0.99^t	0.95^t
1	0.99	0.95
2	0.9801	0.9025
4	0.960596	0.814506
8	0.922745	0.66342
16	0.851458	0.440127
32	0.72498	0.193711
64	0.525596	0.037524

Hva trenger vi for å estimere neste tilstand og belønning?

- Hvis vi kun trenger å vite om den nåværende tilstanden og handlingen, sier vi at problemet har Markov-egenskapen

$$P(r_t = r', s_{t+1} = s' | s_0, a_0, r_0, \dots, r_{t-1}, s_t, a_t) =$$

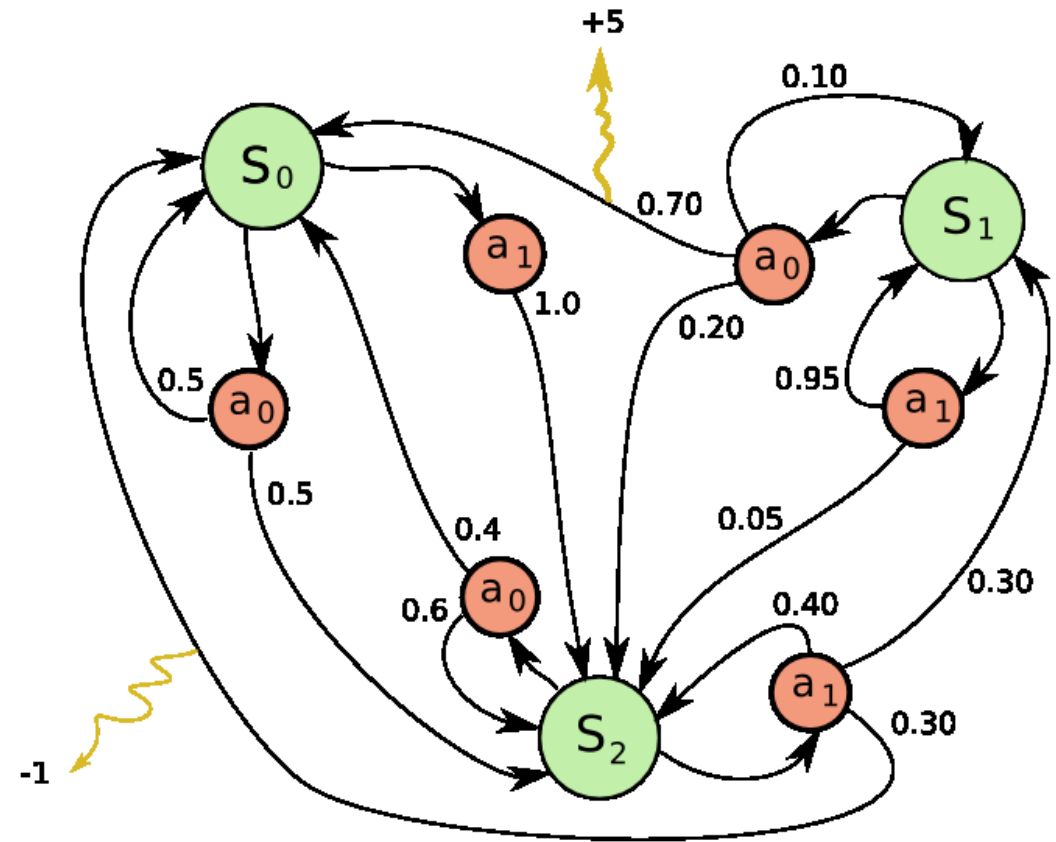
$$P(r_t = r', s_{t+1} = s' | s_t, a_t)$$

- En annen måte å se det på:
Problemet vårt krever ikke at vi husker alt vi har gjort før for å ta valget vi skal ta nå.



Markov-beslutningsprosess (eng: Markov Decision Process – MDP)

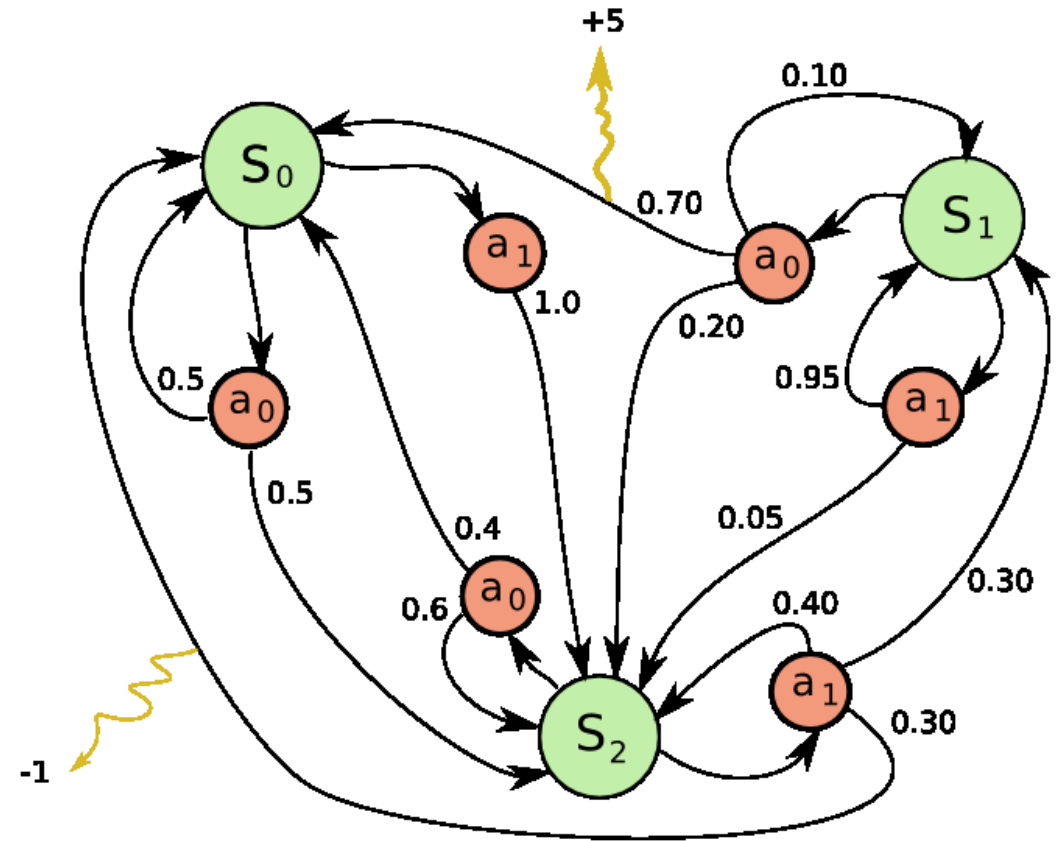
- Det er vanlig å **modellere interaksjonen mellom agent og miljø** som såkalte **Markov beslutningsprosesser**
- Disse representerer et problem som et tuppel (S, A, P_a, R_a) , der
 - S er mengden av mulige tilstander
 - A er mengden av ulike handlinger
 - $P_a(s, s')$ er sannsynligheten for at handling a i tilstand s tar oss til s' i neste tidssteg
 - $R_a(s, s')$ er belønningen vi får når vi tar handling a i tilstand s



Markov-beslutningsprosess (eng: Markov Decision Process – MDP)

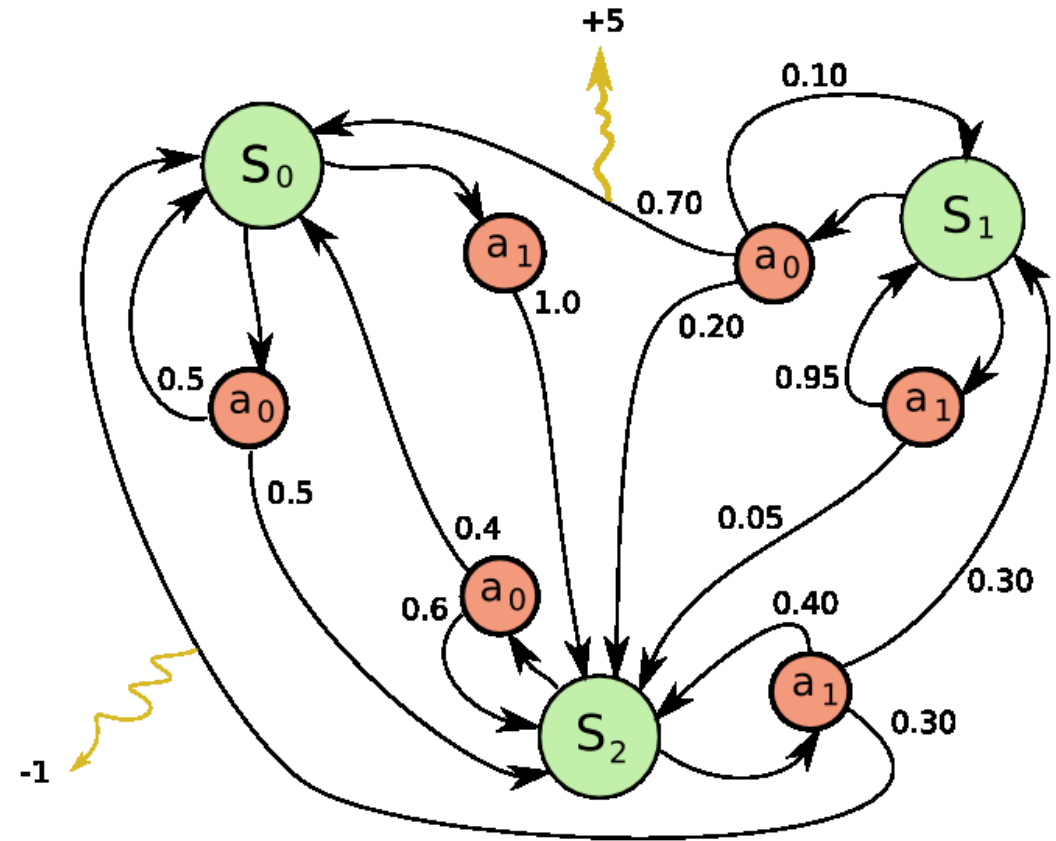
- Målet i forsterkende læringsalgoritmer er å **finne/lære** en “**policy**”, som er en **strategi for å velge den beste handlingen** i hver tilstand – altså den som gir mest belønning totalt sett
- Policy benevnes med π :

$$\pi(s, a) = \Pr(A_t = a \mid S_t = s)$$



Markov-beslutningsprosess (eng: Markov Decision Process – MDP)

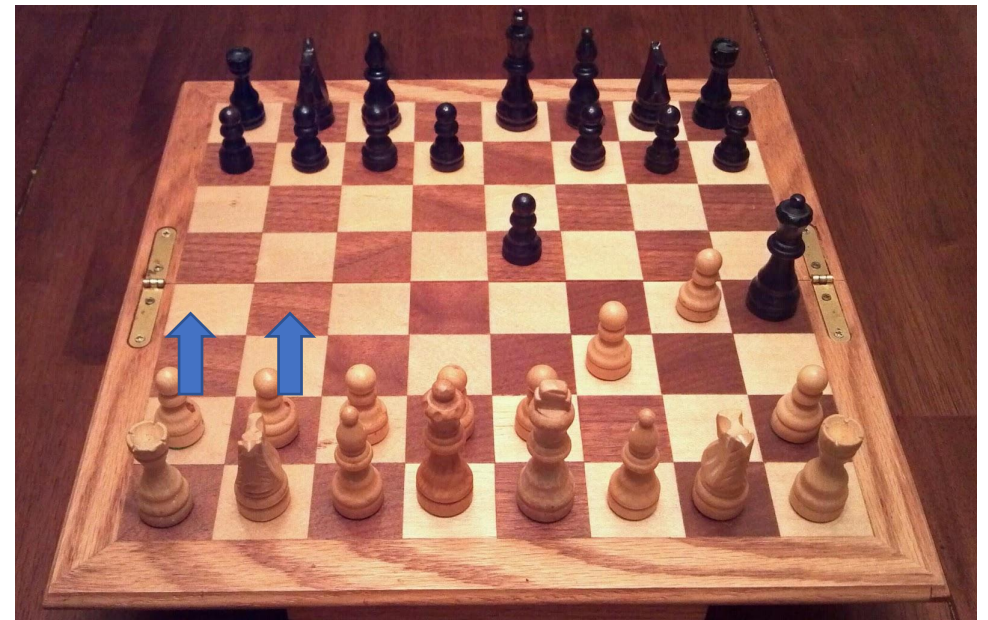
- Som et steg på veien mot en god policy, kan det være lurt å lære gode **estimer** av hvor **nyttige ulike tilstander og handlinger** er – hvilken **verdi** de har.



V: «Denne tilstanden på brettet er verdt 100»

Verdi

- Forventet fremtidig belønning kaller vi *verdi* (eng: *value*)
- Verdi kan beregnes på **to ulike måter**:
 - Verdien til tilstand (gjennomsnitt over alle mulige handlinger i tilstanden): $V(s)$
 - Verdien til par av (tilstand, handling): $Q(s,a)$
- Q og V er ukjente, målet med læringen er å **lære estimer** av disse ved å **utforske miljøet**.
- Hva er fordelene med å lære V istedenfor Q? Og motsatt?



Q: Handling 1 i denne tilstanden er verdt 100, handling 2 er verdt 150, osv...

Q-læring

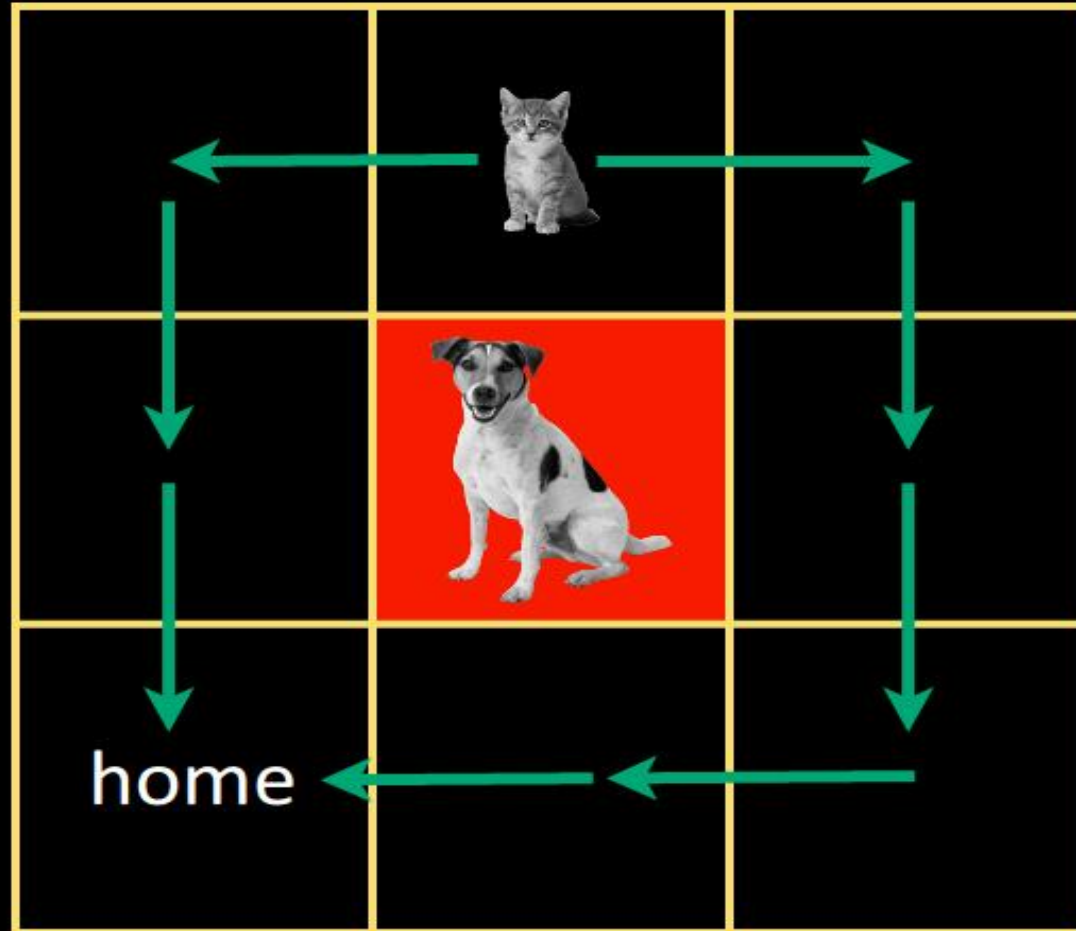
- Vi lærer oss disse verdiene ved å «sende tilbake» verdier fra nåværende tilstand til den forrige

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{\overbrace{r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a)}^{\text{lært verdi}}}_{\substack{\text{belønning} \quad \text{diskonteringsfaktor} \quad \text{estimat av optimal fremtidig verdi}}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

Q-læring - eksempel

- Laget av: Arjun Chandra

toy problem



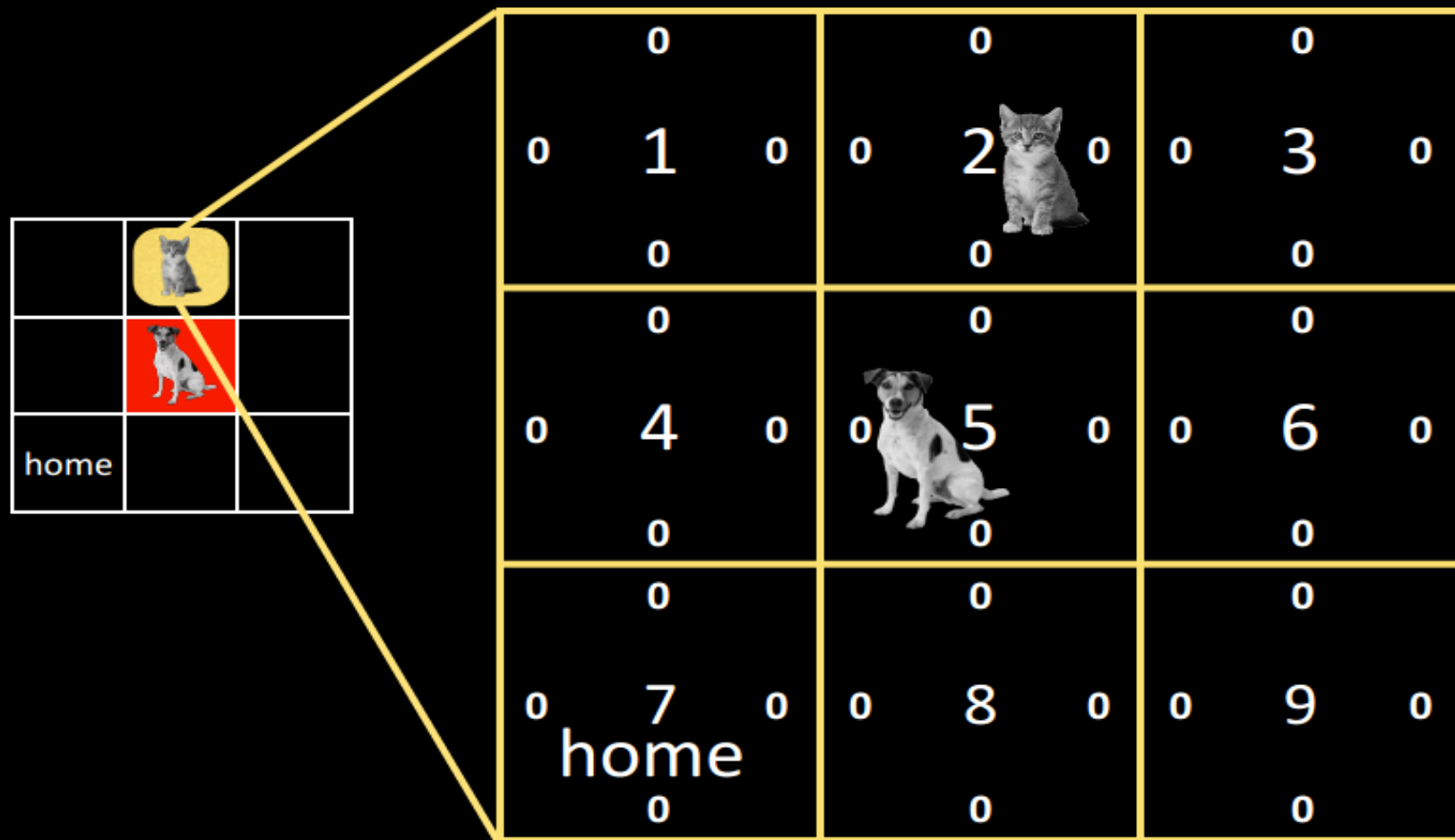
expected long term value of taking
 some action in each state,
 under some action selection scheme?



E{R}	E{R}	E{R}
E{R} E{R}	E{R} E{R}	E{R} E{R}
E{R}	E{R}	E{R}
E{R}	E{R}	E{R}
E{R} E{R}	E{R} E{R}	E{R} E{R}
E{R}	E{R}	E{R}
E{R} h E{R}	E{R} E{R}	E{R} E{R}
E{R}	E{R}	E{R}

our toy problem

lookup table



reward
structure?



	0			0			0	
0	1	0	0	2	0	0	3	0
	0			0			0	
0	4	0	0	5	0	0	6	0
	0			0			0	
0		0		0			0	
0	7	0	0	8	0	0	9	0
	home			0			0	

move...

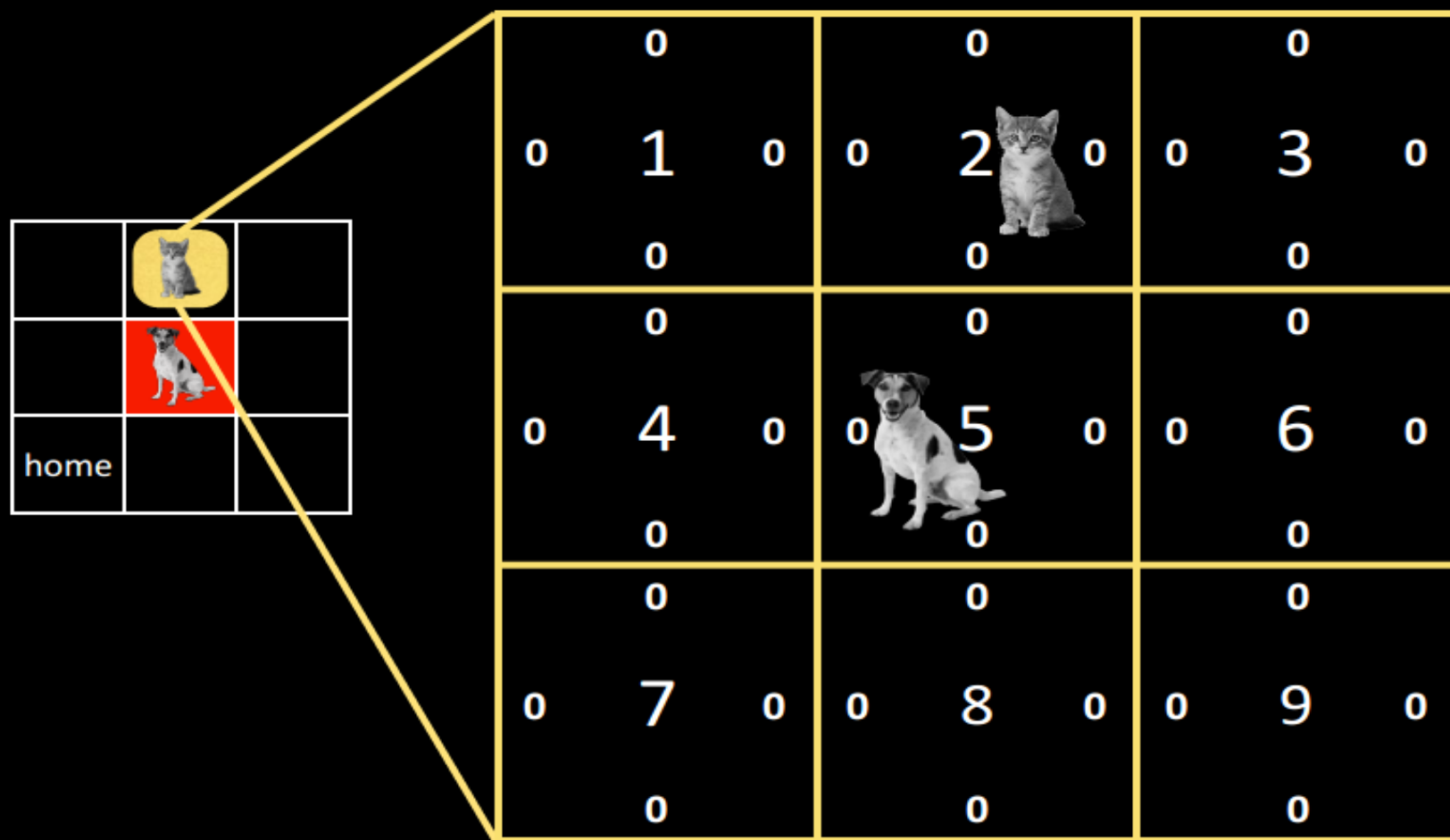
to any cell except 5 and 7:
-1

out of bounds:
-5

to 5:
-10

to 7/home:
10

let's fix $\mu = 0.1$, $\gamma = 0.5$





episode 1 begins...



$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\overbrace{\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}}}_{\text{lært verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

0 0 1 0 0	0 -0.1 2 0 0	0 0 3 0 0
0 0 4 0 0	0 0 5 0 0	0 0 6 0 0
0 0 7 0 home 0	0 0 8 0 0	0 0 9 0 0

0 0 1 0 0	0 -0.1 2 0 0	0 0 3 0 0
0 0 4 0 0	0 0 5 0 0	0 0 6 0 0
0 0 7 0 home 0	0 0 8 0 0	0 0 9 0 0



$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{\overbrace{r_{t+1} + \gamma \max_a Q_t(s_{t+1}, a)}^{\text{lært verdi}}}_{\substack{\text{belønning} \quad \text{diskonteringsfaktor} \quad \text{estimat av optimal fremtidig verdi}}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

-0.5 0 1 0 0 	0 -0.1 2 0 0	0 0 3 0 0
0 0 4 0 0	0 0 5 0 0 	0 0 6 0 0
0 0 7 0 home 0	0 0 8 0 0	0 0 9 0 0

-0.5 0 1 0 0 0	0 -0.1 2 0 0 0	0 0 3 0 0 0
0 0 4 0 0 0	0 0 5 0 0 0	0 0 6 0 0 0
0 0 7 0 home 0	0 0 8 0 0 0	0 0 9 0 0 0

-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
? 	-1 0	0 0
0 4  0	0 0 5 0	0 0 6 0
0	0	0
0 7 0	0 8 0	0 9 0
home	0	0

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
-0.1	0	0
0	0	0
0 4 0	0 5 0	0 6 0
0	0	0
0	0	0
0 7 0	0 8 0	0 9 0
home	0	0

	-0.5			0			0	
0	1	0	-0.1	2	0	0	3	0
	-0.1			0		-10	0	
0	4	?	0	5	0	0	6	0
	0			0			0	
0	7			0			0	
home	0		0	8	0	0	9	0
				0			0	

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

-0.5 0 1 0 -0.1	0 -0.1 2 0 0	0 0 3 0 0
0 0 4 -1 0	0 0 5 0 0	0 0 6 0 0
0 0 7 0 home 0	0 0 8 0 0	0 0 9 0 0

-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
-0.1	0	0
0	0	0
0 4 -1	0 5 0	0 6 0
0	0	0
0	0	0
0 7 0	0 8 0	0 9 0
home	0	0

-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
-0.1	0	0
0	0	0
0 4 -1	0 5 0	0 6 0
0	?	-1
0	0	0
0 7 0	0 8 0	0 9 0
home	0	0

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

-0.5 0 1 0 -0.1	0 -0.1 2 0 0	0 0 3 0 0
0 0 4 -1 0	0 0 5 0 -0.1	0 0 6 0 0
0 0 7 0 home 0	0 0 8 0 0	0 0 9 0 0

-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
-0.1	0	0
0	0	0
0 4 -1	0 5 0	0 6 0
0	-0.1	0
0	0	0
0 7 0	0 8 0	0 9 0
home		
0	0	0



-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
-0.1	0	0
0	0	0
0 4 -1	0 5 0	0 6 0
0	-0.1	0
0	10	0
0 7 0	? 8 0	0 9 0
home	0	0

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

let's work out the next
episode, starting at
state 4

go WEST and then SOUTH

how does the table change?

-0.5	0	0
0 1 0	-0.1 2 0	0 3 0
-0.1	0	0
0	0	0
-0.5 4 -1	0 5 0	0 6 0
1	-0.1	0
0	0	0
0 7 0	1 8 0	0 9 0
0	0	0



$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

and the next episode,
starting at state 3

go WEST -> SOUTH -> WEST -> SOUTH

how does the table change?

	-0.5			0			0	
0	1	0	-0.1	2	0	-0.1	3	0
	-0.1			-1			0	
	0			0			0	
-0.5	4	-1	-0.05	5	0	0	6	0
	1.9			-0.1			0	
	0			0			0	
0	7	0	1	8	0	0	9	0
	0			0			0	



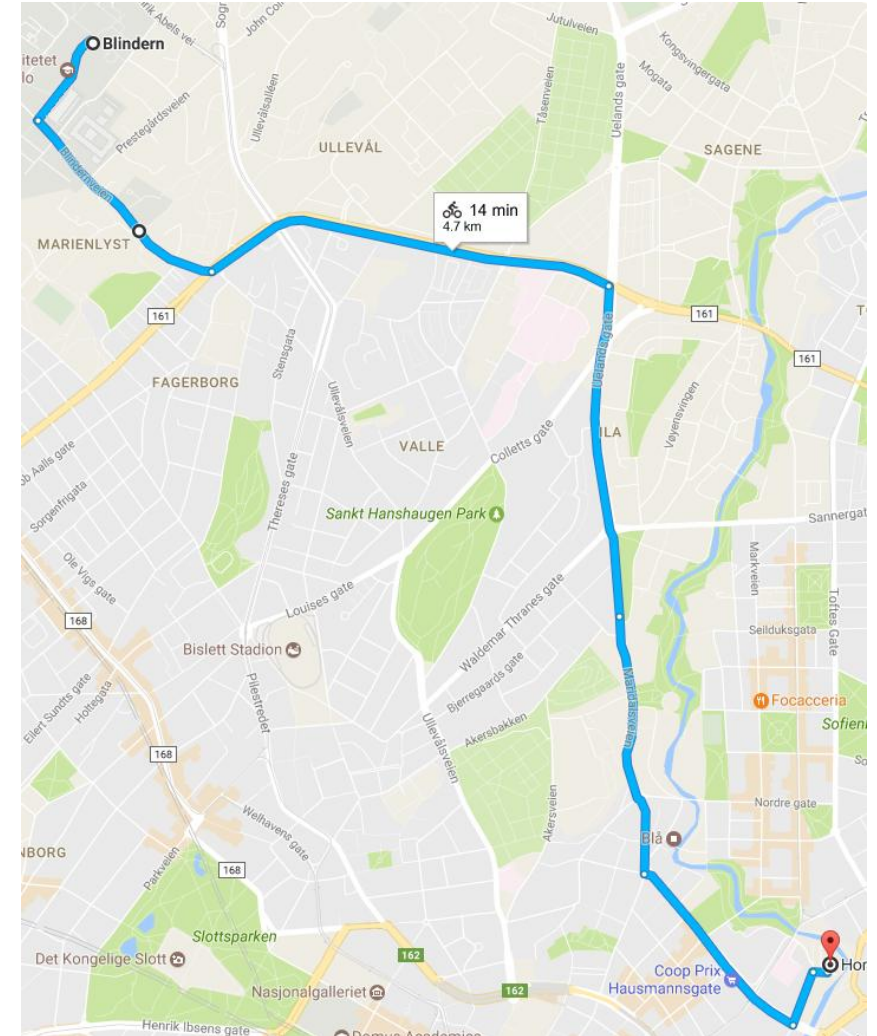
$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

Hvordan velge handling?

- Vi har funnet et estimat for verdi av hver handling: $Q(s,a)$
- Men hvilken handling skal vi egentlig velge? Er det **alltid lurt å velge den med høyest verdi?**
- Kan dere tenke dere situasjoner der det er lurt å velge noe annet?

Utforsking og utnyttning (eng: exploration and exploitation)

- Et generelt problem i alle metoder for optimalisering eller læring
- Bør vi **prøve noe helt nytt** og kanskje lære noe veldig nyttig (**utforske**) eller **holde oss til det trygge** som vi vet fungerer (**utnytte**)?



Handlingsvalg (eng: Action selection)

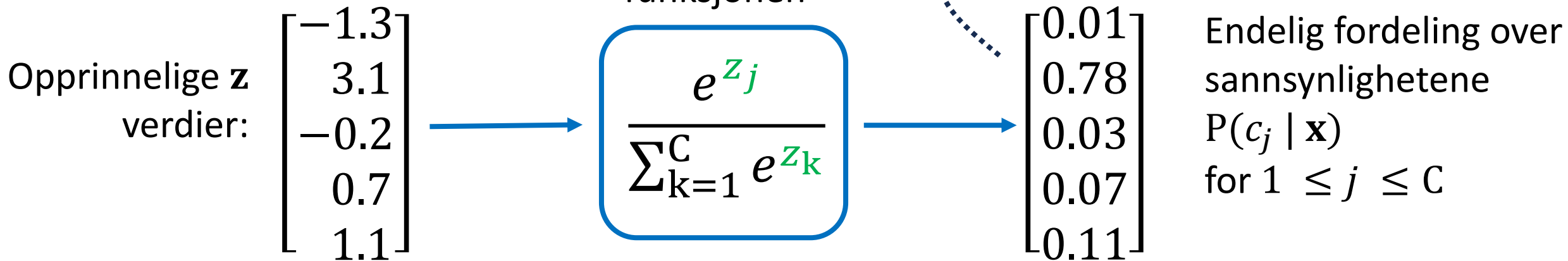
- Gitt de lærte Q-verdiene, hvordan bruker vi dem til å velge handling?

- Grådig: $\text{handling} = \arg \max_a Q(s, a)$

- ε -grådig:
$$\text{handling} = \begin{cases} \arg \max_a Q(s, a) & \text{med sannsynlighet } 1 - \varepsilon \\ \text{tilfeldig handling} & \text{med sannsynlighet } \varepsilon \end{cases}$$

- Softmax:
$$\text{handling} = \begin{cases} a_1 & \text{med sannsynlighet } \frac{e^{Q(s,a_1)}}{\sum_{k=1}^n e^{Q(s,a_k)}} \\ \dots & \dots \\ a_n & \text{med sannsynlighet } \frac{e^{Q(s,a_n)}}{\sum_{k=1}^n e^{Q(s,a_k)}} \end{cases}$$

Softmax-påminner



- softmax har samme rolle som sigmoid funksjon, bare med flere enn 2 klasser: den brukes til å “komprimere” resultatene fra de vektede summene til en sannsynlighetsfordeling
- softmax er en viktig funksjon i maskinlæring, og brukes i mange andre modeller enn logistisk regresjon!

Softmax i handlingsvalg

- Konverterer de ulike Q-verdiene for hver handling til en sannsynlighetsfordeling, slik at høyere Q-verdi gir høyere sannsynlighet for å bli valgt

- Softmax:

$$\text{handling} = \begin{cases} a_1 & \text{med sannsynlighet} & \frac{e^{Q(s,a_1)}}{\sum_{k=1}^n e^{Q(s,a_k)}} \\ \dots & & \\ a_n & \text{med sannsynlighet} & \frac{e^{Q(s,a_n)}}{\sum_{k=1}^n e^{Q(s,a_k)}} \end{cases}$$

<https://web.archive.org/web/20221226013628/http://awjuliani.github.io/exploration/index.html>

Hvordan ville du justert graden av tilfeldighet når du lærer?

Policy

- Det agenten lærer, altså **strategien den har lært seg å følge**, kaller vi for dens **policy**, som benevnes med symbolet π

$$\pi(s, a) = \Pr(A_t=a \mid S_t=s)$$

- Policy avhenger av agentens **lærte verdier (Q eller V)** og **hvilken metode den bruker for å velge handlinger** basert på disse verdiene (grådig, ϵ -grådig, osv.)

På-policy vs av-policy (On-policy vs off-policy) læring

- Q-læring (av-policy):

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\overbrace{\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}}}_{\text{lært verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

- Sarsa (på-policy):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \mu[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Av-policy læring

- Q-læring (av-policy):

$$Q_{t+1}(s_t, a_t) = \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} + \underbrace{\mu}_{\text{læringsrate}} \times \left[\overbrace{\underbrace{r_{t+1}}_{\text{belønning}} + \underbrace{\gamma}_{\text{diskonteringsfaktor}} \underbrace{\max_a Q_t(s_{t+1}, a)}_{\text{estimat av optimal fremtidig verdi}}}_{\text{lært verdi}} - \underbrace{Q_t(s_t, a_t)}_{\text{gammel verdi}} \right]$$

Oppdateringen av Q-verdien inneholder en **antagelse om at vi velger den mest verdifulle handlingen** i neste tilstand.

Q-verdiene vil dermed være **representative for en agent som kun utnytter – aldri utforsker**

På-policy læring

- Sarsa (på-policy):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \mu[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

Oppdateringen av Q-verdien inneholder en **antagelse om at vi følger vår faktiske policy** i neste tilstand (derav på-policy).

Hvis vår policy har en blanding av utforsking og utnyttning (som er vanlig) vil Q-verdiene vil være **representative for en agent som bade utnytter og utforsker**

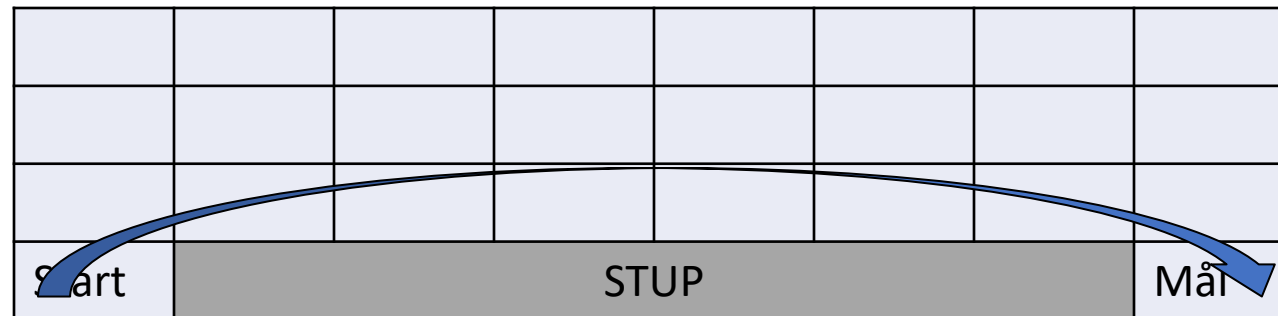
På-policy vs av-policy læring

- Belønningsstruktur: Hvert trekk: -1. Falle i stupet: -100.
- Policy: 90% sjans for å ta handlingen med høyest Q-verdi (utnytting).
10% sjanse for tilfeldig handling (utforsking).

Start	STUP						Mål

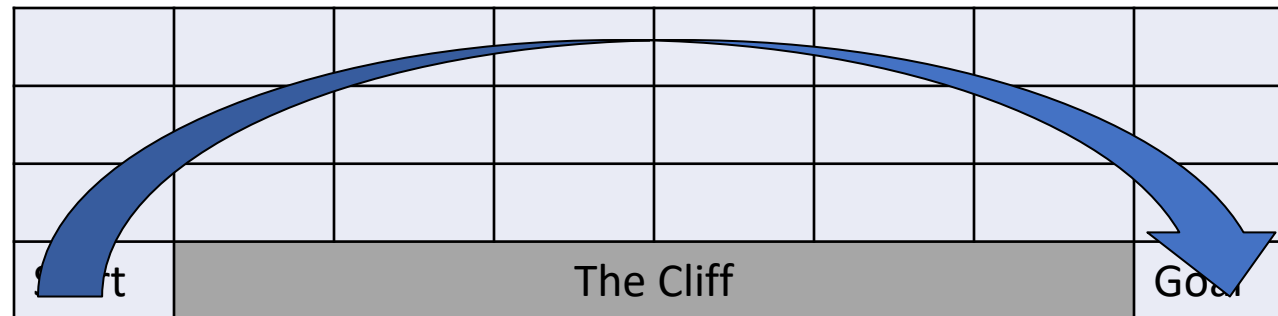
På-policy vs av-policy læring: Q-læring

- Q-læring **antar vi tar den mest verdifulle handlingen i neste tilstand**. Vi lærer oss dermed at det er “trygt” å være nær stupet, siden å **hoppe ned der ikke er en verdifull handling**.
- Resultat-policy er **risikabel, men effektiv**.



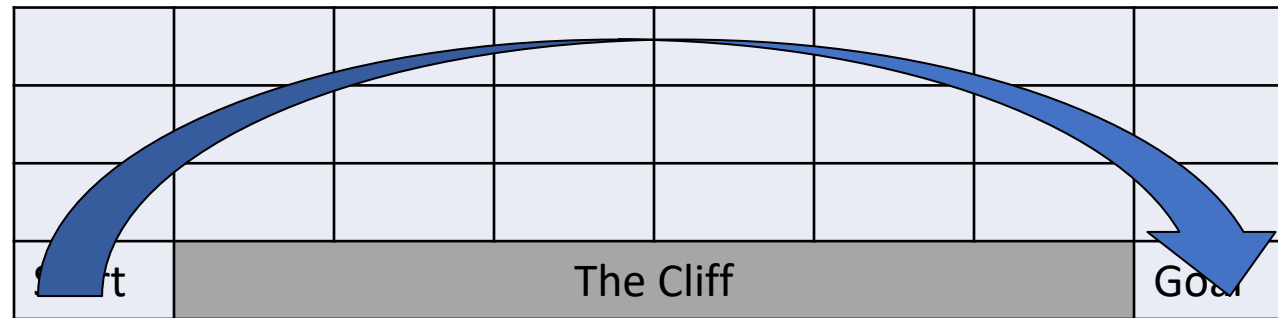
På-policy vs av-policy læring: sarsa

- sarsa **antar at handlingen i neste tilstand følger vår faktiske policy.** Siden vår policy har tilfeldighet i seg (10% sjans for utforsking), fanger q-estimatene opp at det å være nær kanten innebærer risiko for en veldig stor negative belønning.
- Resultat-policy er **tryggere, men mindre effektiv.**

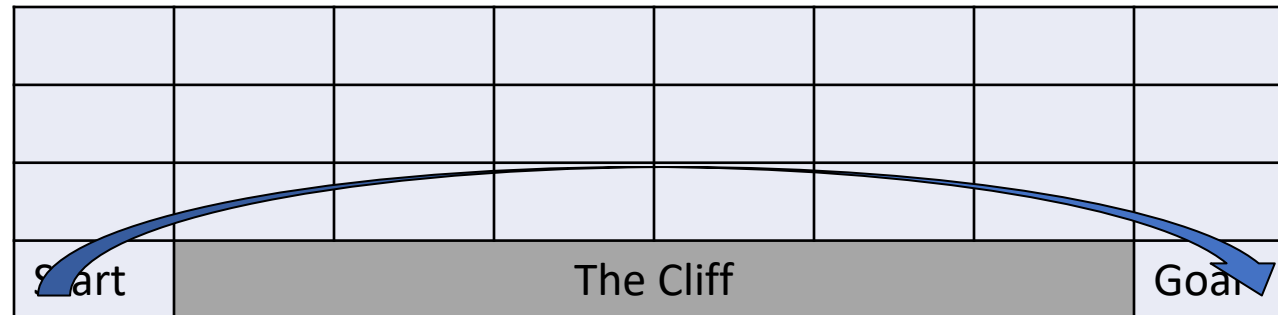


Hvilken plan er best?

- sarsa (på-policy):



- Q-læring (av-policy):

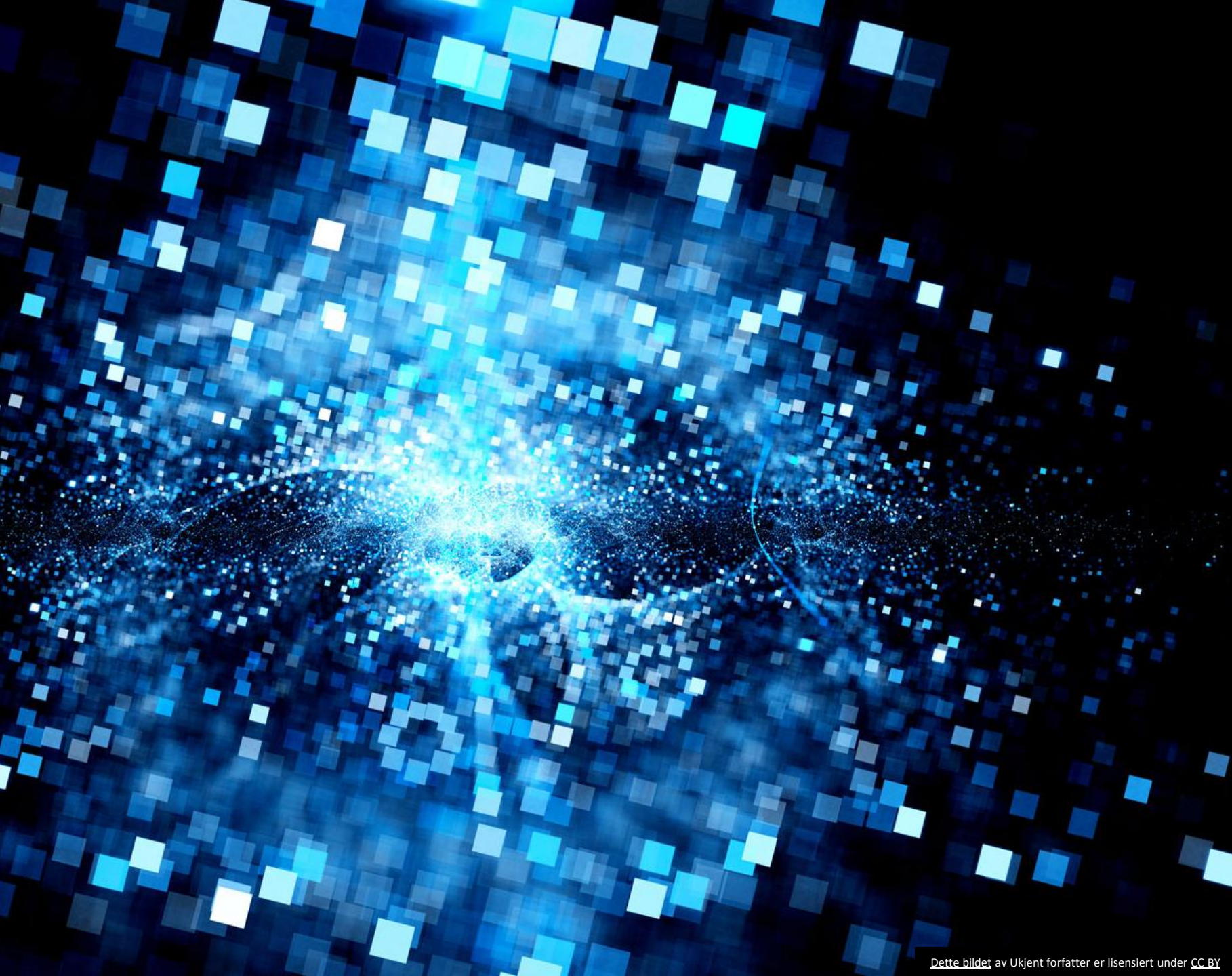


Grid World Demo

- <https://distill.pub/2019/paths-perspective-on-value-learning/>

Oppsummering

- Forsterkende læring kan være en god modell for å **formulere og løse mange problemer fra den virkelige verden**: Å spille spill, kjøre en bil, kontrollere produksjonen på en fabrikk, etc.
- En viktig forskjell fra andre typer ML er at i forsterkende læring er **agenten selv aktiv i å velge sine «treningsdata»** - og den må derfor finne en god balanse mellom å **utforske og utnytte**
- **Q-læring og sarsa** er metoder for å **lære verdier til ulike handlinger i ulike tilstander**, som en stor «tabell» - mer moderne metoder lærer denne sammenhengen i et stort nevralt netteverk i stedet. Slik **dyp forsterkende læring** kan løse mye mer komplekse problemer.



Spørsmål?